UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO



FACULTAD DE INGENIERÍA



MODELADOS DE PROGRAMACIÓN ORIENTADA A OBJETOS

REPORTE DE CÓDIGO: PRÁCTICA 10

REPORTE DE PRÁCTICA: POLIMORFISMO, CLASES ABSTRACTAS Y MOLDES

ALUMNO: MIGUEL ANGEL MEDINA REYES

IKER ALEJANDRO VELASCO GOMEZ

PROFESOR: OSCAR MANUEL RUIZ HURTADO

SEMESTRE: 2027-1

1. FUNDAMENTOS TEÓRICOS

El Polimorfismo de Inclusión es una de las columnas fundamentales de la Programación Orientada a Objetos. Permite tratar a objetos de distintas clases derivadas a través de una misma interfaz o tipo base común (clase abstracta), ejecutando la implementación específica de cada subclase según el objeto concreto alojado en memoria (mecanismo conocido como Ligadura Dinámica o Late Binding).

Conceptos Clave de la Práctica 10:

Clase Abstracta (`Herramienta`): Define la estructura común e impone contratos mediante métodos abstractos que no poseen implementación (`grito()` y `golpear()`). No puede ser instanciada directamente con `new` .

Upcasting (Molde Hacia Arriba): Asignación implícita de una subclase a una variable de tipo base (ej. `Herramienta h = new Arco(15);`). Es 100% seguro y automático.

Downcasting (Molde Hacia Abajo): Conversión explícita de un tipo base a su tipo derivado real (ej. `Arco a = (Arco) h;`). Requiere verificación previa mediante `instanceof` para evitar fallos en tiempo de ejecución.

2. DESARROLLO DE LA PRÁCTICA

Parte A	`Herramienta.java` , `Arco.java` , `Personaje.java` , `Main.java`	Se convirtió `Herramienta` en clase abstracta con métodos `grito()` y `golpear()` . Se creó la clase hija `Arco` y se adaptó `Personaje.atacar()` para delegar el cálculo del daño al método polimórfico de la herramienta. Se eliminó el paquete `Inventario` .
Parte B	`Espada.java` , `Baston.java`	Se implementaron las clases derivadas de `Herramienta` . La `Espada` realiza ataques que desgastan durabilidad (`usar()`), mientras que el `Baston` utiliza daño directo por magia sin consumir durabilidad física (`getDanio()`).

Parte C	Comparar.java	Se demostró el poder del polimorfismo sustituyendo una estructura compleja condicional `if-else` equipada con múltiples operadores `instanceof` por una sola línea polimórfica: `imprimir(h, h.grito(), h.golpear());` .
Parte D	Moldes.java	Análisis de visibilidad según la variable de referencia y verificación de seguridad al forzar conversiones de tipo en tiempo de compilación y ejecución.

3. RESPUESTAS AL CUESTIONARIO DE BITÁCORA

Q1: ¿Quién decide qué métodos se pueden llamar sobre una variable de objeto?

Respuesta: El tipo de la variable de referencia (definido en tiempo de compilación). El compilador Java solo revisa las firmas de métodos declaradas en la clase del molde de la variable. Si una variable está declarada como `Herramienta h` , el compilador solo permitirá llamar a métodos de `Herramienta` , incluso aunque la memoria contenga un objeto `Arco` con métodos propios como `getFlechas()` .

Q2: ¿Quién decide qué código ejecutable específico se corre cuando se hace la llamada?

Respuesta: El tipo real del objeto instanciado en memoria (en tiempo de ejecución / runtime). Mediante la ligadura dinámica, la Máquina Virtual de Java (JVM) busca el método sobrescrito (`@Override`) en la clase real concreta a la que pertenece el objeto.

Q3: ¿Por qué el polimorfismo elimina la necesidad de evaluar `instanceof` y condicionales?

Respuesta: Porque traslada la responsabilidad de decidir qué comportamiento ejecutar desde el código cliente (el juego/controlador) hacia cada objeto individual. El cliente simplemente envía el mensaje (ej. `golpear()`) y cada subclase responde con su propio comportamiento especializado.

4. BITÁCORA DE ERRORES ENCONTRADOS Y SOLUCIONES

Durante el desarrollo de la práctica se experimentaron tanto errores contextuales de entorno/sintaxis como errores provocados deliberadamente para comprender las restricciones de los moldes de tipo.

Error 1: Ubicación Incorrecta al Ejecutar el Comando de Compilación (Consola)

Mensaje del compilador: `error: Invalid filename: src/mazmorra/*.java`

Causa técnica: La terminal se encontraba posicionada dentro del subdirectorio `.../src/mazmorra/` . Al ejecutar `javac -d out src/mazmorra/*.java` , la consola buscaba la ruta relativa `src/mazmorra/src/mazmorra/*.java` , la cual no existe.

Solución aplicada: Se ejecutó el comando de navegación `cd ../..` para regresar a la raíz del proyecto (`base-p09` o carpeta del proyecto) antes de invocar la compilación.

Error 2: Estructura de Llaves Anidadas y Declaración Duplicada en `Comparar.java` Mensaje del compilador: `error: illegal start of expression / unnamed classes are a preview feature`

Causa técnica: Al implementar la Parte C, se duplicó la firma del método `conPolimorfismo()` dentro de sí misma por un descuido en las llaves `{ }` del archivo, rompiendo el balance sintáctico del archivo de Java.

Solución aplicada: Se reestructuró el archivo reemplazando el bloque por una única declaración limpia del método con su ciclo `for-each` correspondiente.

Error 3 (Experimento Parte D1): Violación de Visibilidad del Molde de Referencia

Código probado: `h.getFlechas();` (donde `Herramienta h = new Arco(15);`) Mensaje del compilador: `error: cannot find symbol - method getFlechas() / location: variable h of type Herramienta`

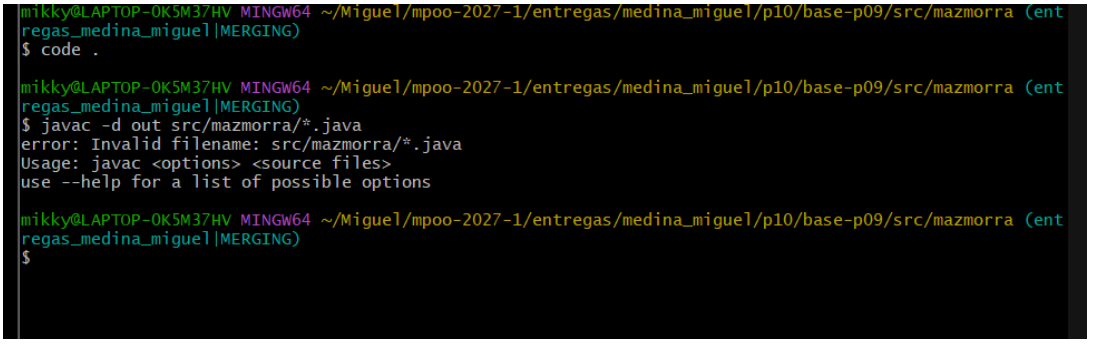
Explicación: Aunque el objeto en memoria es un `Arco` , la variable de referencia `h` es de tipo `Herramienta` . Como `Herramienta` no tiene definido el método `getFlechas()` , el compilador bloquea la llamada previniendo accesos no autorizados.

Error 4 (Experimento Parte D2): Downcast Inválido en Tiempo de Ejecución

Código probado: `Espada e = (Espada) h;` (donde `h` apunta a un objeto `Arco`) Resultado: Compila correctamente, pero al correr arroja `java.lang.ClassCastException: class mazmorra.Arco cannot be cast to class mazmorra.Espada`

Explicación: El compilador permite el molde porque `Espada` es hija de `Herramienta` . Sin embargo, en tiempo de ejecución, la JVM detecta que la instancia real es un `Arco` y rechaza convertirlo en una `Espada` , abortando el programa.

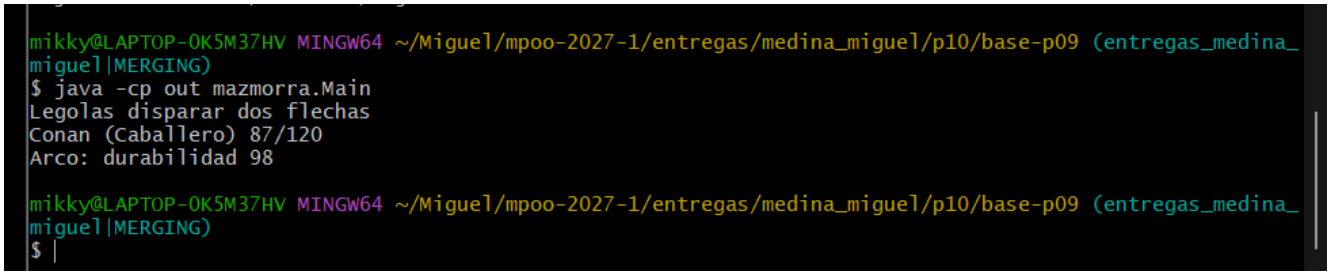
5. EVIDENCIA GRÁFICA DE ERRORES TEÓRICOS (PARTE D)



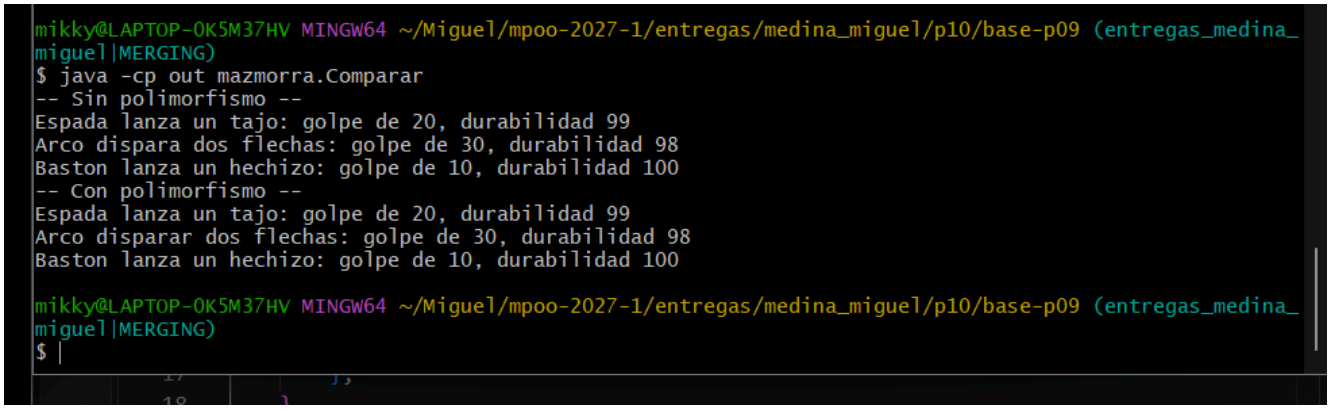
6. EVIDENCIA DE EJECUCIÓN CORRECTA

A continuación se presentan los espacios calibrados para colocar las capturas de pantalla de la terminal que demuestran el funcionamiento correcto de las pruebas requeridas en la práctica.

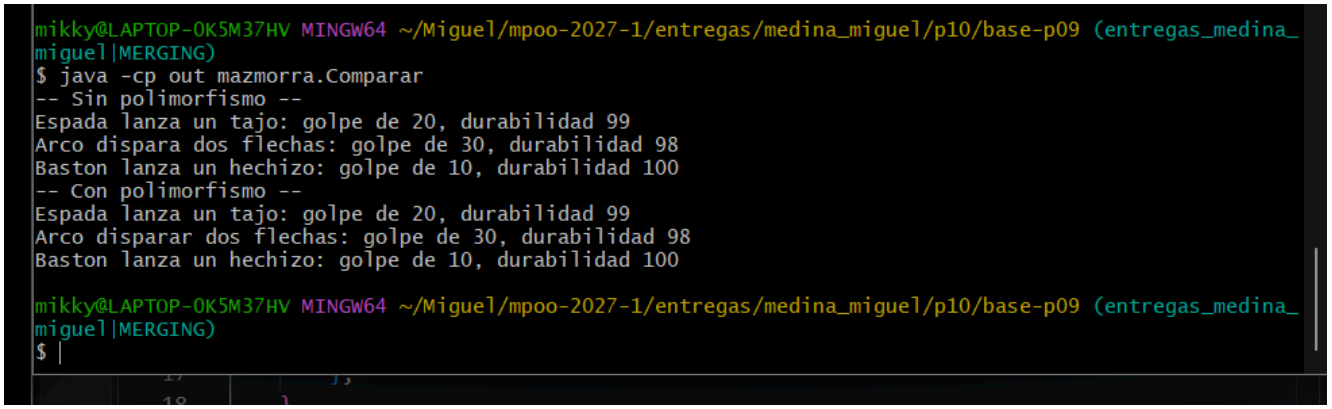
Prueba 1: Ejecución de Main.java (Parte A - Legolas y Arco)



CAPTURA DE TERMINAL: JAVA -CP OUT MAZMORRA.MAIN



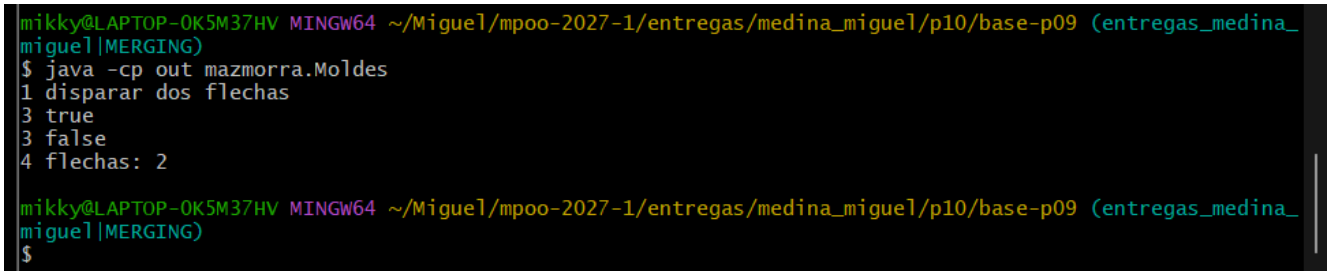
CAPTURA DE TERMINAL: `JAVA -CP OUT MAZMORRA.COMPARAR`



Debe demostrar que los bloques "-- Sin polimorfismo --" y "-- Con polimorfismo --" imprimen exactamente la misma salida en pantalla.

Prueba 3: Ejecución de `Moldes.java` (Parte D - Funcionamiento Normal de Moldes)

CAPTURA DE TERMINAL: `JAVA -CP OUT MAZMORRA.MOLDES`



Muestra los resultados de upcast implícito, `instanceof` (true / false) y downcast explícito a `Arco`.

7. CONCLUSIÓN

La implementación exitosa de la Práctica 10 ha permitido constatar el valor arquitectónico del polimorfismo. Al delegar el comportamiento especializado a cada subclase abstracta, logramos escribir código extensible, limpio y desacoplado, reduciendo drásticamente la fragilidad del sistema al evitar el uso masivo de estructuras condicionales y verificaciones mecánicas de tipo.